LAPORAN PRAKTIKUM
TEKNOLOGI IOT II

Praktikum – 4 Komunikasi Asinkron dan WebSockets

DOSEN PENGAMPU: Dr. Ir. Septafiansyah Dwi Putra S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom.

Disusun oleh:

Nama: Cahya Rindi Arouffeti

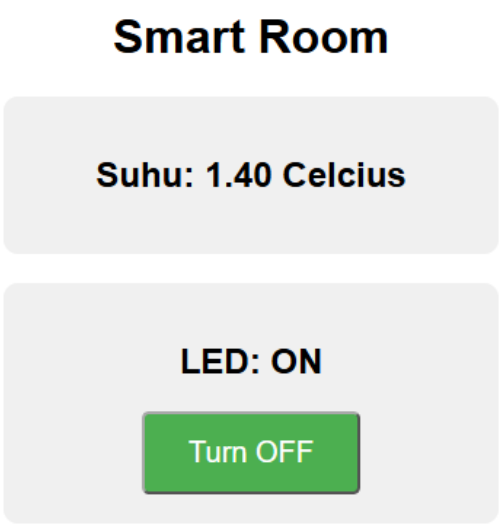
NPM: 25782035

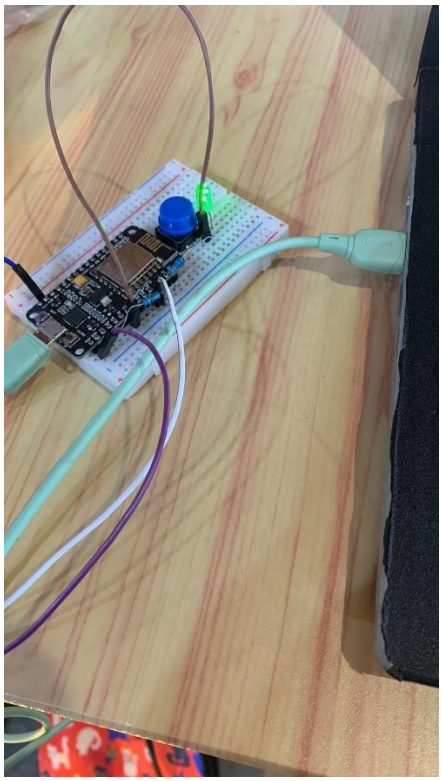
Kelas: TRI 3B

PROGRAM STUDI TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG
TAHUN 2026

4.5 Praktikum

4.5.1 Praktikum 1: *Real-Time Dashboard* & Sinkronisasi Tombol Fisik





Latihan 1 (Teori Jaringan): Jelaskan dengan kata-kata Anda sendiri, mengapa teknologi koneksi *WebSockets* jauh lebih hemat konsumsi data/bandwidth internet dan tidak membuat layar berkedip jika dibandingkan dengan metode *Meta Refresh HTTP* HTML yang Anda gunakan pada Tugas Praktikum Modul 3?

WebSockets lebih hemat bandwidth karena koneksi antara browser dan NodeMCU dibuat sekali dan dipertahankan tetap terbuka. Dengan demikian, keduanya dapat bertukar data tanpa harus membuat permintaan HTTP baru secara berulang. WebSockets juga memungkinkan browser memperbarui bagian tertentu pada halaman tanpa memuat ulang seluruh halaman.Sebaliknya, Meta Refresh HTTP memuat ulang halaman secara berkala. Proses tersebut dapat menggunakan lebih banyak bandwidth karena browser harus meminta dan memproses kembali halaman. Pemuatan ulang halaman juga dapat menyebabkan layar berkedip.

Latihan 2 (Analisis Waktu/Timer): Apa bahaya sistematis yang akan terjadi pada komunikasi *WebSockets* (reaksi antara *browser* dan NodeMCU) jika Anda bersikeras nekat memanggil sintaks `delay(3000)` di dalam siklus fungsi `void loop()` pada draf arsitektur kompleks ini?

Penggunaan `delay(3000)` di dalam fungsi `void loop()` akan menghentikan sementara eksekusi program selama tiga detik. Akibatnya, NodeMCU terlambat memproses pesan browser, memperbarui data sensor, dan menangani komunikasi WebSockets.

```
void loop() {
  ws.cleanupClients();

unsigned long previousMillis = 0;
const unsigned long interval = 3000;

void loop() {
    webSocket.loop();

    unsigned long currentMillis = millis();

    if (currentMillis - previousMillis >= interval) {
        previousMillis = currentMillis;

        // Baca sensor dan kirim data terbaru.
    }
}
  // 1. Eksekusi perangkat keras LED
  digitalWrite(ledPin, ledState ? HIGH : LOW);

  // 2. Baca Tombol Fisik (Debounce) tanpa nge-freeze program
  int reading = digitalRead(buttonPin);
  if (reading != lastButtonState) {
    lastDebounceTime = millis();
  }
  if ((millis() - lastDebounceTime) > debounceDelay) {
    if (reading != buttonState) {
```

Latihan 3 (Modifikasi Front-End): Analisislah fungsi JavaScript pada blok `// Merombak UI Tombol & Teks sesuai status hardware`. Nama ID elemen HTML (`<span id="...">`) apa saja yang menjadi target injeksi/pengubahan teks oleh *script* yang ditulis oleh Anda?

```
      break;
  }
}

document.getElementById("ledStatus").innerText = data.led;
document.getElementById("tempValue").innerText = data.suhu;
document.getElementById("humValue").innerText = data.hum;
```

Latihan 4 (Pengembangan Telemetri Sensor - Kelembapan): Pada Praktikum 1, antarmuka web hanya menyajikan data suhu (*temperature*). Modifikasilah program agar sistem dapat membaca dan menampilkan data kelembapan (*humidity*) secara *real-time* via WebSockets:

```
String currentHum = "--";

unsigned long previousMillis = 0;
const unsigned long interval = 3000;

void notifyClients() {
    String json = "{";
    json += "\"led\":\"" + ledStatus + "\",";
    json += "\"suhu\":\"" + currentTemp + "\",";
    json += "\"hum\":\"" + currentHum + "\"";
    json += "}";

    webSocket.broadcastTXT(json);
}

void loop() {
    webSocket.loop();

    unsigned long currentMillis = millis();

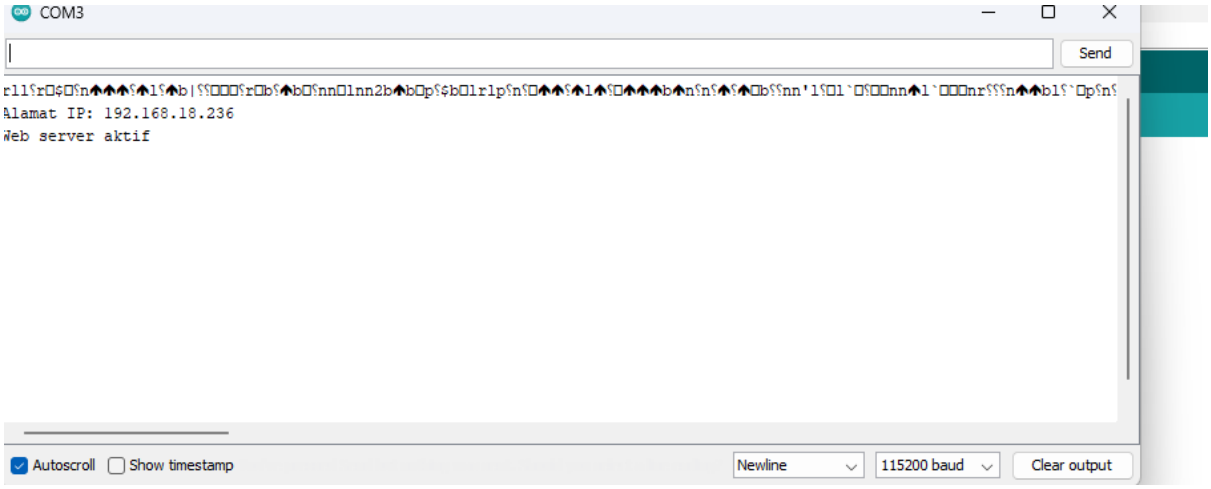
    if (currentMillis - previousMillis >= interval) {
        previousMillis = currentMillis;

        float temperature = dht.readTemperature();
        float humidity = dht.readHumidity();

        if (!isnan(temperature)) {
```

4.7 Tugas: Sistem Kendali Intensitas Cahaya (PWM Slider) via *WebSockets*

Sinyal Digital (*ON/OFF*) sangat berguna, namun bagaimana jika Anda ditugaskan merancang antarmuka penyesuai intensitas redup-terang lampu (*Dimmer*) dari *smartphone*?



```
#include <WiFiClientSecure.h>
#include <WiFiClientSecureBearSSL.h>
#include <WiFiServer.h>
#include <WiFiServerSecure.h>
#include <WiFiServerSecureBearSSL.h>
#include <WiFiUdp.h>

#include <ESP8266WiFi.h>
#include <ESP8266WebServer.h>

const char* ssid = "BRO'WN CAFE";
const char* password = "browncafe";

ESP8266WebServer server(80);

const int pinLED = D1;
int nilaiPWM = 0;

const char index_html[] PROGMEM = R"rawliteral(
<!DOCTYPE html>
<html>
<head>
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Smart Lighting</title>
</head>
<body>
  <h2>Smart Lighting</h2>



  <p>Kecerahan LED:
    <span id="nilai">0</span>
  </p>

  <input type="range" min="0" max="1023" value="0"
    style="width: 100%;"
    oninput="ubahLED(this.value)">

  <script>
    function ubahLED(nilai) {
      document.getElementById("nilai").innerHTML = nilai;

      fetch("/pwm?nilai=" + nilai);
    }
  </script>
</body>
</html>
)rawliteral";

void handleRoot() {
  server.send_P(200, "text/html", index_html);
}

void handlePWM() {
  if (server.hasArg("nilai")) {
    nilaiPWM = server.arg("nilai").toInt();
    nilaiPWM = constrain(nilaiPWM, 0, 1023);
```

```
  pinMode(pinLED, OUTPUT);
  analogWriteRange(1023);
  analogWrite(pinLED, 0);

  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);

  Serial.print("Menghubungkan WiFi");

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println();
  Serial.print("Alamat IP: ");
  Serial.println(WiFi.localIP());

  server.on("/", handleRoot);
  server.on("/pwm", handlePWM);

  server.begin();
  Serial.println("Web server aktif");
}

void loop() {
  server.handleClient();
}
```